



نظري

1

7

840 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Flynn's classification & Proposals Complexity

د. روان قرعوني



برمجة تفرعية

RB Informatcs; 14/05/2025

محتوى مجاني غير مخصص للبيع التجاري

السلام عليكم ورحمة الله وبركاته



عدنا إليكم بقاء جديد ومادة جديدة بعنوان البرمجة التفرعية التي ستكون مختلفة عن طريقة التفكير في البرمجة التقليدية نأمل أن تكون مفيدة وممتعة لكم. نسأل الله أن نكون على قدر المسؤولية، وأن نقدم لكم محتوى يليق بتطلعاتكم ويساهم في تطوير مهاراتهم.

في البداية لتتعرف على البرمجة التفرعية ؟ في حال وجود برنامج طويل يأخذ 10 ثواني يتم تنفيذه ، هنا وجد أن الاعتماد على البرمجة التسلسلية المعروفة لا يجدي نفعاً بشكل دائم ، مثل في حال مسائل البيانات الكبيرة جداً والتي تحتاج إلى زمن تنفيذ كبير، نكون بحاجة للحصول على النتائج بسرعة أكبر ومن هنا ظهرت الحاجة الى البرمجة التفرعية حيث يتم استخدام أكثر من معالج على التوازي لضمان السرعة .

ماذا سنتعلم من خلال هذه المادة ؟ كيف سنقسم مسألة لعدة أجزاء. وإعطاء كل جزء لمعالج يعمل على الجزء الخاص به لوحده. يتبقى لدينا مهمة جمع النتائج للحصول على النتيجة النهائية

لندعم فهمنا بمثال عن الفرق بين البرمجة التسلسلية والتفرعية

لدي 10 عناصر، كل عنصر تأخذ معالجته ثانية، في لبرنامج التسلسلي سنحتاج 10 ثواني .

فلو أحضرت 10 أجهزة و وزعت كل عنصر على جهاز، كم سيأخذ معي من الوقت ؟ بالتفكير نعتقد أنه سوف يأخذ ثانية واحدة ولكن في الحقيقة لن يأخذ ثانية واحدة فقط ، بسبب زمن الارسال والاستقبال ومنه قد يصبح زمن التنفيذ ثانيتين وليست واحدة فقط، مع ذلك يبقى أفضل من الزمن التسلسلي (10 ثواني) .

لذلك نستنتج ان التفرعية هي محاولة توزيع العمل على عدة أجهزة (معالجات)، حيث كل جهاز يقوم بالعمل الخاص به، ومجموع النتائج يكون النتيجة المرادة و بأسرع وقت ممكن.

بعض الملاحظات المهمة التي يجب فهمها ومراعاتها

- بالبرمجة التفرعية لا يمكن الوصول دوماً لزمن أمثل بالمعالجة ، بسبب زمن التوزيع والتجميع .
- ليست كل البرامج قابلة للتفرع بشكل كامل. (ممكن أن يتم التوزيع بشكل جزئي إذ لا بد من وجود بعض الأجزاء التي تحتاج للعمل بشكل تسلسلي).
- زيادة عدد المعالجات لا تعني دائماً الحصول على نتيجة أفضل .
- نحن نحصل على سرعة مقابل زيادة الـ Hardware، ومنه يوجد كلفة عند العمل بشكل تفرعي.
- وليست كل الشركات قادرة على دفع هذه التكاليف.
- ليس لكل المعالجات نفس القدرة ، إذ لا يمكن دوماً تقسيم العمل على المعالجات بشكل متساوي .
- الـ Threads ليست برمجة تفرعية. بل هي توهماً بالعمل كذلك، فهي تعمل على نفس المعالج ولكن بشكل متوازي .
- إدارة الـ Threads صعبة كما تعمل كلها على نفس الذاكرة ومنه يوجد تضارب البيانات إذا لم يتم برمجتها بشكل صحيح .
- أما في البرمجة التفرعية فقد تم الفصل ، حيث لكل معالج ذاكرته الخاصة. ولا مانع من وجود الـ Threads ضمن البرنامج الواحد .
- الفرق بين البرمجة التفرعية و الـ Threads مثلاً عندما نقوم بإنشاء 2 threads فإن التنفيذ سيكون جزء من الـ thread الأول يليه جزء من الـ thread الثاني ثم نكرر (أي أن تنفيذ التعليمات لا يتم في الوقت نفسه كما يظهر لنا) ، بينما في البرمجة التفرعية تكون العملية مختلفة حيث أن التنفيذ يكون في الوقت نفسه على التوازي

أنواع الحواسيب التفرعية

■ SMMP : (Multiprocessor) shared memory

ما هو؟ نظام يحتوي على ذاكرة واحدة مشتركة بين عدة أنوية/معالجات.
مثل حالة الـ multicore ، كل نواة تعتبر معالج مستقل ولكن كلهم يعملون على ذاكرة واحدة.

مشكلتها: التنافس على الموارد والتضاربات بسبب العمل على ذاكرة وحدة، لأن المعلومة تكون موجودة للجميع.
إدارة الوصول للذاكرة والمعلومات الموجودة فيها هو أهم شيء عند العمل بها.

مميزتها: عدم الاضطرار لإرسال البيانات لكل معالج ليقوم بمعالجتها وبالتالي زيادة السرعة (كل نواة تصل إلى البيانات مباشرة من الذاكرة المشتركة) .

مثال: لدي صورة أريد معالجتها وهي موجودة على ذاكرة واحدة ويمكن لكل نواة أخذ جزء منها، مثلاً تم تقسيمها لأربع أقسام وعندي أربع أنوية، إذا كانت الأربع معالجات تعمل بشكل منفصل عندها سأضطر لإرسال كل قسم منها لمعالج فيوجد هنا زمن إرسال وزمن استقبال النتائج. أما عند وجودهم جميعاً على الـ shared memory عندها الصورة موجودة على الذاكرة وكل نواة تأخذ قسمها مباشرة.

DMMP: (Multiprocessor) Distributed memory

ما هو؟ لدي لكل معالج ذاكرة خاصة به و يعمل بدون تضارب.

سليبيتها: تكلفة إرسال المعلومات.

تذكرة: ال thread عند العمل على معالج واحد، توهم المستخدم بالعمل على التفرع رغم أنها تعمل على التسلسل وذلك بسبب إمكانية عملها على التوازي. لا يتم توزيع العمل على المعالجات بشكل متساو إذ عند وجود معالج قوي أعطيه مهمة أكبر من غيره. عند وجود مصفوفة تحتاج 4 ثواني عند العمل على معالج واحد.

سؤال؟ هل إذا وزعتها على أربع معالجات ستأخذ ثانية واحدة؟ لا، بسبب وجود زمن إرسال وزمن استقبال. فممكّن أن تأخذ ثانيتين عوضاً عن الأربع ثواني.

ومنه لا يمكن الوصول لزمن مثالي بسبب:

- زمن الإرسال والاستقبال.
- عدم قابلية البرنامج للتفرع (عملية تعتمد على نتيجة عملية أخرى).

$$\frac{\text{الزمن التسلسلي}}{\text{عدد المعالجات}} = \text{الزمن المثالي}$$

أنواع البرامج التفرعية

وهي أن يتم تصنيف البرامج التفرعية لعدة أنواع بناءً على آلية التطبيق !!
والمقصود بـ "آلية التطبيق" كيفية توزيع ال Data وال Code على المعالجات.
فببساطة، يفهم ذلك من خلال معرفة ما إذا كانت ال Data وال Code الموزعة على المعالجات هي نفسها لجميعها (جميع المعالجات) أم لا ولهذا تم التصنيف لأربعة أنواع SIMD, SIMD, MIMD, MISD
نفسها لجميعها (جميع المعالجات) أم لا ولهذا تم التصنيف لأربعة أنواع SIMD, SIMD, MIMD, MISD

1. SISD (single instruction single data)

تعلّمة واحدة على نفس ال data. ولكن تنفيذها على عدّة معالجات. أي أن كل معالج ينفّذ نفس الكود على نفس ال data ولكن على التوازي.
حالات استخدامها قليلة ولكنها ممكنة.

2. SIMD (single instruction multiple data)

تعلّمة واحدة على بيانات متعددة.
مثل الصور، أي لو أردت تغيير البكسلات بصورة ما فأنا أزيد كل بكسل (يكونون على شكل مصفوفة) فرضاً بمقدار 2، فنلاحظ أن تعلّمة الزيادة نفسها لكل معالج ولكن على عدّة بيانات أي أن كل معالج يستخدم بيانات مستقلة عن الآخر.

3. MIMD (multiple instructions multiple data)

تعليمات متعددة على بيانات متعددة
فأنا أرسل لكل معالج الكود الخاص فيه والبيانات التي عليه أن يعمل عليها.

4. MISD (multiple instructions single data)

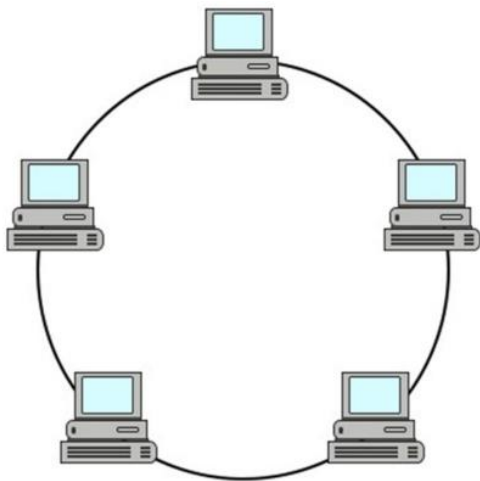
تعليمات متعددة على بيانات واحدة (نادر التطبيق).
مثل: مصفوفة واحدة أريد أن أنفذ عليها أكثر من عملية على التوازي.

أنواع شبكات الوصل

للوصل بين الأجهزة المختلفة طرق مختلفة ولها محور أساسي وهو القطر Diameter أي المسافة من أول حاسب لآخر حاسب.

يوجد 4 أنواع Ring ,Tree, 2D array, Hypercube

1. Ring



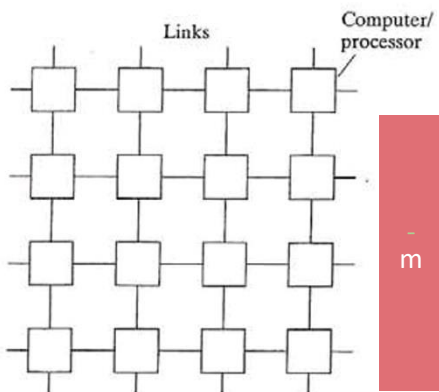
أي لدي ربط حلقي بين الحواسيب.
ولها نوعين: باتجاه واحد ، باتجاهين (أي أن الجهاز يستطيع التواصل مع الجهاز الذي بعده والجهاز الذي قبله).

أطول مسافة في حال الاتجاه الواحد هو n

أطول مسافة في حالة الاتجاهين: $n/2$

حيث n هي عدد الحواسيب

2. 2D array



أربط الحواسيب على شكل مصفوفة وتكون المسافة بين أبعد نقطتين هي $m+n$



من عمل بما علم، أورثه الله علم ما لم يعلم



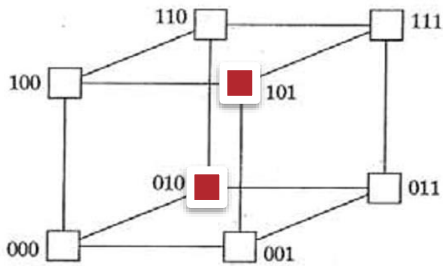
Hypercube .3

يعد لوغاريتمي أيضاً.

ويحوي فرضاً 8 عقد فالمسافة بين أبعد عقدتين فيه:

هي لوغاريتم عدد العقد

$$\log(n)$$



Tree .4

أفضل أنواعها الشجرة الثنائية التامة، ميّزتها أن ارتفاعها لوغاريتمي.

إذا كان لدي 8 أوراق إذا ارتفاعها 3 والذي هو $\log(8) = 3$

الارتفاع هو الحد الأدنى من لوغاريتم العدد العقدي،

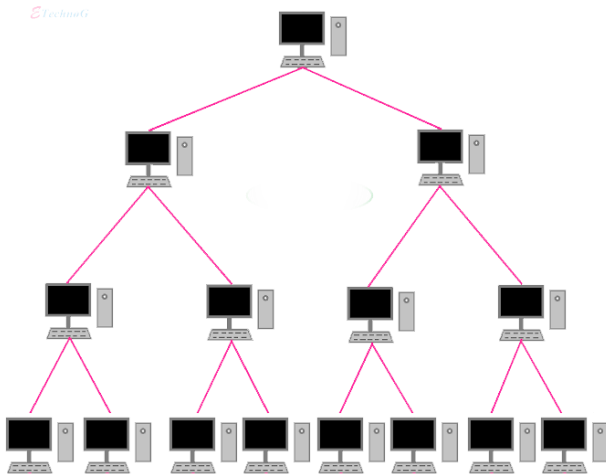
و لها قانونين:

- الحد الأدنى من قانون الـ n حيث n هو عدد العقد

$$\text{Log}(15) = 3 \text{ فالحد الأدنى هو } 3$$

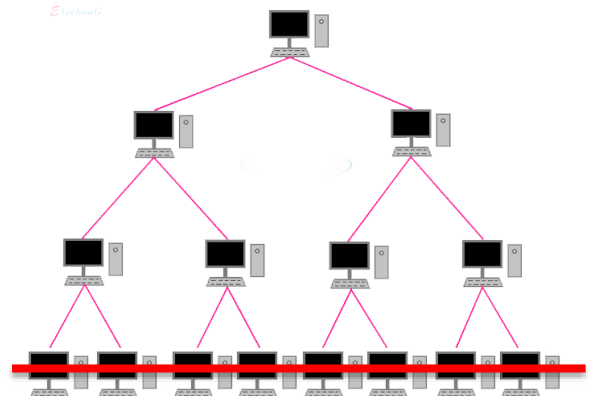
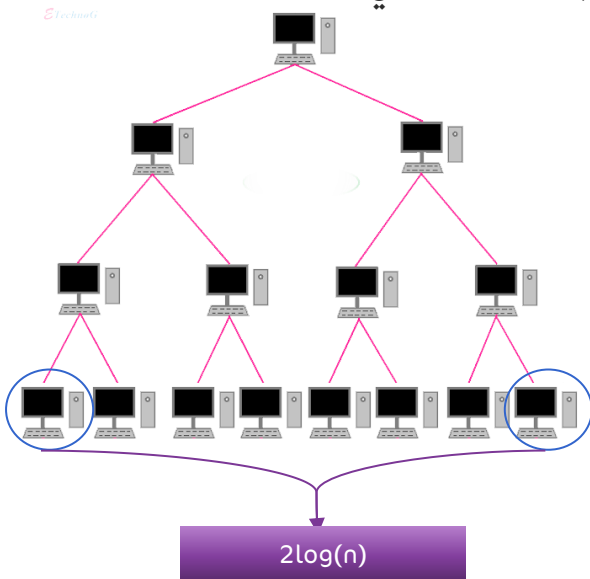
- أن تكافئ الـ f ، حيث f هو عدد الأوراق.

$$\text{Log}(8) = 3$$



المسافة بين أول عقدة وآخر عقدة هي $2 \log(n)$
ويعتبر أوفر من الحلقي

بإزالة السطر النهائي يمكن القول أن ارتفاع الشجرة:
 $\text{Log}(4) = 2$ أو $\log(7) = 2$ والحد الأدنى هو 2



قوانين ومعاملات لقياس عمل البرنامج التفرعي

1- معامل الاستجابة لمعالج: Speed up factor

$$\text{Speedup factor} = s(p) = \frac{ts}{tp}$$

S(p): معامل الاستجابة

ts: time serial ، الزمن التسلسلي

tp: time parallel ، الزمن التفرعي

بالحالة المثالية أحسن قيمة يمكن الوصول لها:

$$tp = \frac{ts}{p}$$

p: عدد المعالجات

أي أن الحالة المثالية هي الزمن التسلسلي نفسه عدد المعالجات

■ مثال: إذا كان الزمن التسلسلي 10 ولدي 10 معالجات يكون أفضل زمن يمكن الوصول له هو $tp = \frac{ts}{p} = 1$

$$s(p) = 10 \text{ ومنه } \frac{10}{10} = 1$$

فإن أحسن قيمة لمعامل الاستجابة تكون $s(p) = p$ ، نفسه عدد المعالجات.أما أسوأ قيمة (الحالة الأسوأ) عندما: الزمن التسلسلي يساوي التفرعي: $tp = ts$

كلما كانت أقرب من الواحد تكون الـ p يكون معامل الاستجابة سيء وكلما كانت أقرب من الـ p يكون معامل الاستجابة أفضل.



كل فكرة عظيمة بدأت خلم صغير في قلب شخص آمن بنفسه.

Efficiency-2 الكفاءة

$$\text{Efficiency} = \frac{ts}{tp * p} * 100\%$$

أفضل حالة:

$$\text{Efficiency} = \frac{ts}{tp * p} * 100\% = \frac{s(p)}{p} * 100\%$$

وإن أفضل حالة لمعامل الاستجابة هي p ومنه

$$\text{Efficiency} = \frac{p}{p} * 100\% = 100\%$$

ومنه أفضل حالة 100% وكلما اقتربت من 0%، كان الأداء أسوأ

3-التكلفة Cost

بالسلسلية: هو ts أي الزمن التسلسلي نفسه، فهي كلفة التنفيذ نفسها.
 بالتفرعية: $p * tp$ أي الزمن التفرعي ضرب عدد المعالجات.

مثال: احسب كلفة برنامج تفرعي يأخذ 3 ثواني ويوجد 10 معالجات.

$$\text{cost} = 3 * 10 = 30$$

استنتاج:

$$\text{cost} = tp * p = \frac{ts}{s(p)} * p = \frac{ts}{E}$$

أفضل كلفة للعمل التفرعي: $\text{cost} = ts$
 فإن أفضل cost أن يكون مثل الزمن التسلسلي

آمن خلمك، حتى وإن شكك فيه الجميع، فالحلم

بداخلك أنت، وليس فيهمز

See you SOON